QR Attendance System - DBMS Project Report

1. Objective

The primary objective of this project is to develop a robust, secure, and automated Exam Attendance System using QR codes. Traditional attendance systems are prone to proxy marking and human error. This system addresses these issues by leveraging a completely normalized Relational Database Management System (RDBMS) paired with a modern web interface. It ensures strict data integrity, handles concurrent scans safely, and automates attendance tracking using advanced database features.

2. Features

- **Role-Based Access Control**: Separate secure portals for Teachers and Students.
- **Automated QR Code Generation**: Students are automatically assigned unique, tamper-proof QR tokens upon registration.
- **Real-Time Scanning**: Teachers can use their device cameras to scan student QR codes.
- **Transaction-Safe Operations**: Concurrent attendance marking is handled via database-level locking to prevent race conditions.
- **Analytics & Reporting**: Real-time views and stored procedures calculate attendance percentages and generate absentee lists.
- **Audit Logging**: All critical database actions (like marking attendance) are automatically logged for security review.

3. Entity-Relationship (ER) Model

The system consists of the following core entities and relationships:

- **TEACHERS**: Can create multiple EXAMS and mark ATTENDANCE.
- **STUDENTS**: Can attend multiple EXAMS.
- **EXAMS**: Hosted by one or more TEACHERS (Invigilators).
- **ATTENDANCE**: The resolving entity for the many-to-many relationship between STUDENTS and EXAMS.

Relationships:

- TEACHER to EXAM: Many-to-Many (Resolved by `exam_invigilators`)
- STUDENT to EXAM: Many-to-Many (Resolved by `attendance`)
- TEACHER to ATTENDANCE: One-to-Many (A teacher marks multiple attendance records)

4. Relational Model & Normalized Relations (3NF)

The schema is normalized to the Third Normal Form (3NF) to eliminate data redundancy and insertion/deletion anomalies.

1. **`teachers`** (`teacher_id` PK, `name`, `email` UNIQUE, `password_hash`, `created_at`)
2. **`students`** (`student_id` PK, `enrollment_number` UNIQUE, `name`, `email` UNIQUE, `password_hash`, `qr_code_text` UNIQUE, `created_at`)
3. **`exams`** (`exam_id` PK, `exam_name`, `exam_date`, `start_time`, `end_time`, `status`, `created_at`)
4. **`exam_invigilators`** (`exam_id` FK, `teacher_id` FK, `assigned_at`) - *Composite PK (`exam_id`, `teacher_id`)*

5. **`attendance`** (`attendance_id` PK, `exam_id` FK, `student_id` FK, `marked_by_teacher_id` FK, `status` , `marked_at` , `remarks`) - *Unique Constraint on (`exam_id` , `student_id`)*
6. **`audit_logs`** (`log_id` PK, `action_type` , `table_name` , `record_id` , `old_value` , `new_value` , `timestamp`)

5. Advanced DBMS Features Implemented

5.1. Triggers

Triggers are used to automate background tasks without application-level intervention.

- **`BEFORE INSERT ON students`** : Automatically generates a unique `UUID()` and assigns it to the `qr_code_text` column whenever a new student is enrolled.
- **`AFTER UPDATE ON attendance`** : Automatically inserts a record into the `audit_logs` table whenever a student's status changes from 'ABSENT' to 'PRESENT', ensuring non-repudiation.

5.2. Stored Procedures & Transactions

Stored procedures encapsulate complex business logic directly inside the database server.

- **`MarkAttendance`** :
  - This procedure accepts the scanned QR code and validates the student and exam status.
  - It uses a **TRANSACTION** (`START TRANSACTION` , `COMMIT` , `ROLLBACK`) to guarantee ACID properties.
  - It utilizes `SELECT ... FOR UPDATE` to exclusively lock the specific attendance row, preventing race conditions if a QR code is accidentally scanned twice in the same millisecond.

5.3. Scalar Functions

Functions are used to compute derived attributes dynamically.

- **`GetStudentAttendancePercentage`** : Takes a `student_id` and calculates their total attendance percentage across all exams by counting rows in the `attendance` table.

5.4. Cursors

Cursors are utilized for procedural row-by-row processing within the database.

- **`GenerateAbsenteesList`** : This procedure uses a cursor to iterate through all students who are marked 'ABSENT' for a specific `exam_id` . It fetches their names and emails one by one and populates a temporary table `temp_absentees` that can be exported for email notifications.

5.5. Views

Views are implemented to simplify complex analytical queries and restrict direct table access.

- **`vw_exam_attendance_summary`** : Joins `exams` and `attendance` tables to provide a real-time aggregate view showing the total number of students present vs. absent for every active exam.

6. Limitations and Future Scope

1. **Screen Capturing**: A student could theoretically screenshot their static QR code and send it to a friend.
   - *Future Scope*: Implement Time-Based One-Time Passwords (TOTP) embedded within the QR code that refresh every 30 seconds.

2. **Hardware Dependency**: Relies on the quality of the invigilator's smartphone/webcam camera to scan codes efficiently in low light.
3. **Network Dependency**: The system requires an active local network or internet connection to communicate with the central database during scanning. Offline syncing is not currently supported.